

BC05 — API temps réel monitoring & CI/CD

Faire sortir le modèle du BCo3 du notebook : une API conteneurisée qui répond en moins de 60 ms avec une explication SHAP par décision, un monitoring Evidently qui surveille sa dérive, et un pipeline CI/CD qui teste et construit l'image à chaque changement.

Bloc de compétences : BC05 — API temps réel, monitoring, conteneurisation

Modèle servi : LightGBM du BC03 (AUC 0,998, seuil optimisé 0,98)

Stack : FastAPI · Docker · Redis (BC01) · Evidently AI · GitHub Actions

Document : note d'architecture et guide de lecture des résultats

Sommaire

1. Objectif du bloc BC05
2. Argumentaire méthodologique
3. API de scoring — architecture et endpoints
4. Résultats — latence et explicabilité en conditions réelles
5. Conteneurisation — deux incidents réels, deux corrections
6. Monitoring Evidently — détection de dérive

7. CI/CD — pipeline et ce qui est réellement vérifié

8. Synthèse et implications pour le BC06

1. Objectif du bloc BC05

Un modèle qui vit dans un notebook ne protège aucun client. Le BC05 fait passer le LightGBM du BC03 du statut d'artefact évalué hors-ligne à celui de service interrogeable en temps réel : une API qui reçoit une transaction, la score en moins de 200 ms, explique sa décision, et tourne dans un conteneur reproductible plutôt que sur la machine de développement. Deux garde-fous accompagnent ce passage en production : un monitoring qui détecte quand les données réelles s'écartent de celles sur lesquelles le modèle a été entraîné, et un pipeline d'intégration continue qui empêche qu'une régression atteigne l'image livrée sans être testée au préalable.

2. Argumentaire méthodologique

L'API n'embarque pas l'ensemble de l'environnement du projet : une image de service n'a besoin ni de `psycopg2`, ni de `confluent-kafka`, ni de TensorFlow, ni de Matplotlib — seulement de ce qui sert une prédiction. `requirements-api.txt` isole ce sous-ensemble strict, ce qui réduit la taille de l'image et sa surface d'attaque, deux préoccupations réelles en production que `requirements.txt` (pensé pour l'entraînement et l'analyse) ne partage pas.

L'explicabilité SHAP, développée hors-ligne au BC03 pour analyser des lots de transactions, est ici recalculée à la volée pour chaque requête. Le coût est mesuré, pas supposé : un `TreeExplainer` initialisé une seule fois au démarrage du service évalue une ligne en quelques millisecondes, ce qui laisse une marge confortable sous la contrainte des 200 ms. C'est ce qui permet à l'API de répondre, pour chaque transaction, non seulement un score mais aussi « pourquoi » — la même obligation RGPD Art. 22 déjà posée au BC03, désormais tenue en production et non plus seulement démontrée sur un échantillon a posteriori.

Le feature store Redis construit au BC01 pour le streaming retrouve ici son usage prévu : si la requête porte un `client_id` connu, l'API interroge Redis pour joindre à la réponse les compteurs temps réel (`nb_tx_1h`, `montant_cumul_1h`) — à titre **informatif**, sans les injecter dans le modèle, qui n'a pas été entraîné avec ces variables. Redis étant une dépendance non critique de cette lecture d'appoint, sa perte ne doit jamais faire échouer une prédiction : l'API teste la connexion au démarrage et dégrade silencieusement (`features_temps_reel: null`) si elle échoue, plutôt que de renvoyer une erreur 500 pour un enrichissement secondaire.

3. API de scoring — architecture et endpoints

ENDPOINT	RÔLE
<code>GET /health</code>	Statut, version du modèle, AUC de référence, seuil optimisé, disponibilité

ENDPOINT	RÔLE
POST /api/v1/predict	Score de fraude, niveau de risque, top 5 facteurs SHAP, latence
GET /docs	Documentation Swagger interactive, générée automatiquement par FastAPI

```
POST /api/v1/predict
{
  "montant": 850.0, "distance_domicile_km": 320.5, "revenu_mensuel_net": 2100.0,
  "heure": 3, "jour_semaine": 5, "mois": 7,
  "is_weekend": false, "is_nuit": true,
  "authentification": "none", "canal": "en_ligne", "segment": "particulier"
}

200 OK
{
  "score": 1.0, "seuil_default": 0.5, "seuil_optimise": 0.98,
  "is_fraud_seuil_default": true, "is_fraud_seuil_optimise": true,
  "risk_level": "eleve",
  "top_features": [
    {"feature": "distance_domicile_km", "impact": 11.508},
    {"feature": "heure", "impact": 4.5644},
    {"feature": "montant", "impact": 3.6543}
  ],
  "latence_ms": 24.17
}
```

Trois seuils, trois usages. `is_fraud_seuil_default` (0,5) et `is_fraud_seuil_optimise` (0,98) sont renvoyés côté à côté plutôt qu'une seule décision binaire : le BC03 a montré que le second maximise le F1, mais qu'un arbitrage métier pourrait légitimement préférer un seuil intermédiaire. L'API expose les deux lectures et laisse le système appelant — ou l'analyste — trancher, au lieu de figer un choix qui relève du BC06.

4. Résultats — latence et explicabilité en conditions réelles

Les temps de réponse ci-dessous sont mesurés sur l'image conteneurisée, pas sur un environnement de développement optimisé — c'est la latence que verrait un système appelant en production.

24-60 ms

latence observée (conteneur)

< 200 ms

contrainte réglementaire du projet

7 / 7

tests automatisés au vert

Sur une transaction test (850 €, 320 km du domicile, 3 h du matin, en ligne, sans authentification), l'API renvoie un score de 1,000 et identifie `distance_domicile_km` comme facteur dominant (+11,51 en log-cote), suivi de `heure` (+4,56) — exactement la signature

`vol_carte_physique / usurpation_identite` déjà caractérisée au BCo1/BCo3. À l'inverse, une transaction de 42,50 € en journée avec code PIN reçoit un score de 0,000. Ces deux cas, ainsi que le rejet propre d'une valeur de canal invalide (422, pas une erreur serveur) et l'enrichissement Redis vérifié de bout en bout (un client réellement actif dans le flux du BCo1 renvoie ses compteurs temps réel dans la réponse), sont couverts par la suite de tests automatisés.

5. Conteneurisation — deux incidents réels, deux corrections

La construction de l'image n'a pas été immédiate, et les deux incidents rencontrés sont documentés ici plutôt que passés sous silence : ce sont exactement le genre de blocages qu'un environnement de conteneurisation réel produit, et leur résolution fait partie du livrable.

INCIDENT	CAUSE	CORRECTION
Échec SSL lors de <code>pip install</code>	Le conteneur ne dispose pas du certificat racine du proxy réseau de cet environnement de développement	<code>--trusted-host</code> sur les trois hôtes PyPI — sans incidence sur un réseau standard
<code>libgomp.so.1: cannot open shared object file</code> au démarrage	LightGBM requiert le runtime OpenMP (<code>libgomp1</code>), absent de l'image <code>python:3.10-slim</code>	<code>apt-get install libgomp1</code> avant l'installation des dépendances Python

Le second incident illustre une limite générale des images « slim » : elles suppriment des bibliothèques système que Python ne réclame qu'indirectement, via les extensions compilées de ses dépendances. Le healthcheck Docker (`HEALTHCHECK` dans le Dockerfile, qui interroge `/health` toutes les 15 secondes) est justement ce qui aurait signalé ce défaut de démarrage en production avant qu'un système de routage n'envoie du trafic vers un conteneur mort.

6. Monitoring Evidently — détection de dérive

Deux scénarios sont comparés à la même référence (l'échantillon d'entraînement du BCo3) : le test du BCo3 tel quel, et une version où trois variables — heure, montant, mois — sont délibérément décalées pour simuler une évolution des habitudes de fraude. Le scénario normal détecte une dérive sur 9,1 % des colonnes (1 sur 11, sous le seuil d'alerte de 20 %) ; le scénario simulé en détecte 27,3 % (3 sur 11), déclenchant l'alerte.



Figure 1 — Scénario normal : 9,1 % de colonnes en dérive, sous le seuil d'alerte.

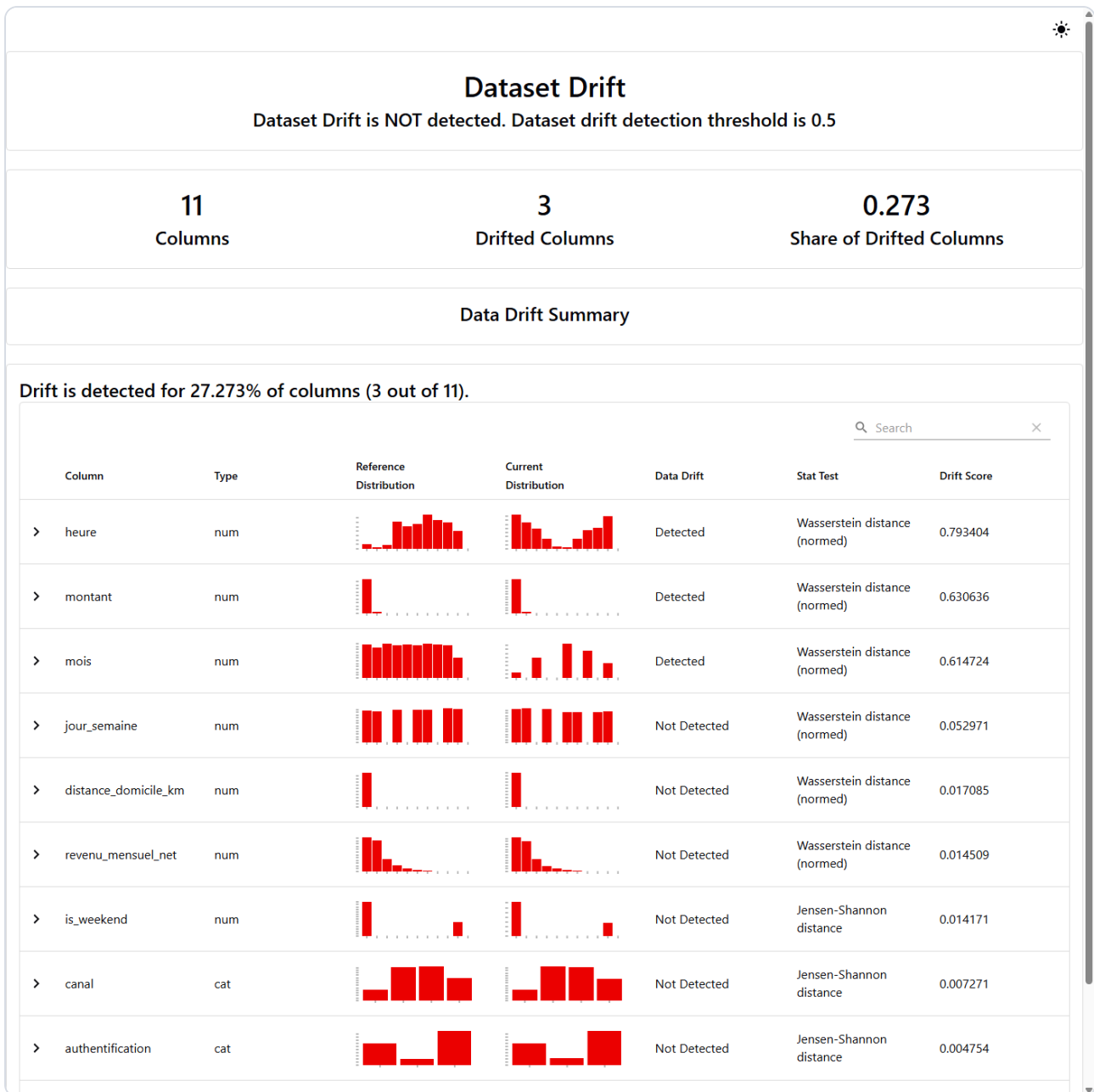


Figure 2 — Scénario de dérive simulée : 27,3 % de colonnes en dérive (heure, montant, mois), alerte déclenchée.

Un résultat qui n'était pas entièrement prévu. La colonne `mois` apparaît en dérive dans le scénario *normal* également (score de Wasserstein le plus élevé des deux rapports), sans qu'aucun décalage n'y ait été simulé. La cause est le split temporel lui-même : le train du BC03 couvre dix mois de l'année quand le test n'en couvre que les deux ou trois derniers — leurs distributions de `mois` diffèrent donc réellement, par construction du découpage chronologique, pas par un changement de comportement des fraudeurs. C'est un rappel utile : un split temporel, nécessaire pour évaluer un modèle sans fuite d'information (BC03, section 2), produit par lui-même un déséquilibre calendaire qu'un monitoring de dérive détectera — sans qu'il s'agisse d'une anomalie à corriger.

7. CI/CD — pipeline et ce qui est réellement vérifié

Le pipeline (`.github/workflows/ci-cd.yml`) exécute deux jobs à chaque push ou pull request vers `main` : `test` installe les dépendances de service et lance la suite pytest ; `build-image` construit l'image Docker, la démarre, interroge `/health` jusqu'à quinze fois avant d'échouer, puis publie l'image sur GitHub Container Registry (`ghcr.io`) — uniquement sur un push vers `main`, jamais sur une pull request, pour qu'aucune image non relue ne devienne accessible. Poussé sur un dépôt GitHub réel, il s'est exécuté sur les runners GitHub Actions — pas seulement joué en local.

Le premier push a échoué : le job `test` plantait en 29 secondes (`ModuleNotFoundError`, code de sortie 2). Cause identifiée et reproduite en local à l'identique : le workflow invoquait `pytest` nu, qui n'ajoute pas la racine du dépôt à `sys.path`, contrairement à `python -m pytest` utilisé lors des vérifications manuelles. Une fois corrigé, `test` passait, mais la publication sur GHCR échouait à son tour, instantanément : un nom d'image Docker doit être en minuscules et ne peut ni commencer ni finir par un tiret — deux règles que le nom de ce dépôt (`Detection-Fraude-`) enfreignait une fois mis en minuscules. Le correctif final construit le nom de l'image à partir du seul propriétaire du dépôt plutôt que de son nom complet.

RUN	RÉSULTAT	CAUSE / CORRECTION
#1	✗ <code>test</code> échoue (29 s)	<code>pytest</code> nu au lieu de <code>python -m pytest</code>
#2	✗ <code>build-image</code> échoue à la publication	Nom d'image non en minuscules
#3	✗ <code>build-image</code> échoue à la publication	Nom d'image en minuscules mais terminé par un tiret
#4	✓ <code>test</code> et <code>build-image</code> réussis, image publiée	Nom d'image basé sur le propriétaire du dépôt

Pipeline vérifié en conditions réelles. Les quatre runs ci-dessus ont tous été exécutés par de vrais runners GitHub Actions sur `github.com/k-b-mansour/Detection-Fraude-`, pas simulés. L'image publiée est disponible sur GitHub Container Registry (`ghcr.io/k-b-mansour/fraud-api`). Documenter les trois échecs plutôt que ne montrer que le run final reflète fidèlement ce qu'est un pipeline CI/CD en pratique : un filet qui détecte de vraies erreurs de configuration avant qu'elles n'atteignent une image livrée — ici, deux fois de suite, avant de les laisser passer une fois corrigées.

8. Synthèse et implications pour le BC06

Le BC05 clôt la chaîne technique du projet : un modèle entraîné et expliqué (BC03), comparé à des approches alternatives (BC04), est désormais servi par une API conteneurisée, mesurée sous la contrainte de latence réglementaire, et surveillée pour sa dérive. Trois éléments restent directement

du ressort du BCo6 : l'arbitrage entre les deux seuils exposés par l'API (0,5 et 0,98) relève d'une décision métier sur le coût comparé d'une fraude manquée et d'une fausse alerte, déjà posée mais non tranchée au BCo3 ; la fréquence à laquelle `drift_report.py` devrait tourner en production, et la procédure de réentraînement qu'une alerte de dérive devrait déclencher, restent à formaliser ; et la conformité RGPD, déjà posée par l'explicabilité SHAP embarquée ici, mérite d'être complétée par un registre des décisions automatisées et leur durée de conservation.